Week 1 - Introduction

Well-Posed Learning Problem

Definition

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

Example A checkers learning problem:

- Task T: playing checkers
- Performance Measure P: percent of games won against opponents
- Training Experience E: playing practice games against itself

Designing a learning program

1. Choose the training experience
2. Choose the target function
3. Choose a representation for the target function
4. Choose a learning algorithm to approximate the target function

Choice 1: the training experience

Feedback type

- direct (for example given a labeled board we are given the best move)
- indirect (only final game outcome is known (*credit assignment* which of many moves deserves credit for win)

Degree of Control - learner has over the sequence of training examples

Representation - How well the distribution of training examples represents the distribution over which performance P will ultimately be measured

Choice 2: The target function

Design a function that lets the program choose the best parameters for its goal. *Note* it is easier to learn an *evaluation function* than to learn how to "play" directly.

Choice 3: The representation

Here we have to learn an **approximation** because it is to computationaly expensive to learn *true* function.

Choice 4: The learning algorithm

Estimate the *value,cost,error* of the current parameters to the true values and adjust accordingly.

Algorithm: LMS

Known as Least Mean Squares, fit the weights so that our approximation best matches the true values.

$$E = \sum_{\langle b, V_{train}(b)\rangle} (V_{train}(b) - \hat{V}(b))^2$$

Intuitively, when our approximation under-estimates the weights are nudged up in proportion to its feature's contribution, and vice-versa.

Final Design - Four Modules

1. Performance System - solves the task
2. Critic - Analyzes the performance system and produces training examples
3. Generalizer - Runs the learning algorithm to update approximation
4. Experiment Generator - Proposes the next problem to improve coverage

Perspectives

Learning, is defined as a search through candidate hypotheses for the one that best fits the training examples and constraints. These *learning algorithms* differ in:

- Hypothesis space being searched (decision trees, neural networks)
- Search strategy used (greedy, gradient descent)

Week 1 - Concept Learning

Definition Concept Learning: inferring a boolean-valued function from training examples of its input and output

We want to learn a target concept(classification) from training examples, by choosing hypotheses from hypothesis space $\mathcal{H}$.

Hypothesis Representation: a conjunction of constraints on attributes. Each is either,

- A specific value (water=warm)
- "¿' - any value is acceptable
- "∅" - no value is acceptable

Definition Inductive Learning Hypothesis: Any hypothesis found to approximate the target function well over a sufficiently large set of training examples will also approximate the target function well over other unobserved examples.

Fact General-to-specific Ordering: For hypotheses h_j, h_k defined over $\mathcal{X}$: h_j is **more_general_than_or_equal_to** h_k (written $h_j \geq_g h_k$) iff

$$\forall x \in \mathcal{X} : \left[(h_k(x) = 1) \rightarrow (h_j(x) = 1)\right].$$

Find-S

Finding a maximally specific hypothesis

FIND-S Algorithm

1. Initialize h to the most specific hypothesis in $\mathcal{H}$.
2. For each positive training instance x:
   - For each attribute constraint a_i in h:
   - If a_i is not satisfied by x, replace a_i with the next more general constraint satisfied by x.
3. Output h.

Intuition: FIND-S ignores negative examples entirely and simply climbs the general-to-specific ordering, generalizing just enough at each positive example to keep covering it

Questions: Does it converge, why prefer maximally specific hypothesis (if not many) if there are others that are equally consistent, what if data is noisy

Version Spaces and Candidate Elimination

Fact A hypothesis h is **consistent** with a set of training examples D iff $h(x) = c(x)$ for every $\langle x, c(x)\rangle \in D$.

The **version space**, $VS_{H,D}$ is the subset of hypotheses from $\mathcal{H}$ consistent with the training examples in D:

$$VS_{H,D} \equiv \{h \in \mathcal{H} \mid \text{Consistent}(h, D)\}$$

Note: List-then-eliminate could work correctly, but it is intractable

Solution: Every member of the (possibly huge) version space is bracketed between two small *boundary sets*:

- G - the set of maximally general members of $VS_{H,D}$
- S - the set of maximally specific members of $VS_{H,D}$

$$VS_{H,D} = \{hH \mid \exists s \in S, \exists g \in G, g \geq gh \geq gs\}$$

S and G implicitly represent every hypothesis in the version space, without listing them.

Candidate-Elimination

Initialize:

$$G \leftarrow \{\text{most general hypothesis in } \mathcal{H}\}, \qquad S \leftarrow \{\text{most specific hypothesis in } \mathcal{H}\}.$$

for all training examples d **do**
  if d is a positive example **then**
    Remove from G any hypothesis inconsistent with d.
    for all $s \in S$ inconsistent with d **do**
      Remove s from S.
      Add to S all minimal generalizations h of s such that:
      h is consistent with d, and some member of G is more general than h.
      Remove from S any hypothesis more general than another in S.
    end for
  else if d is a negative example **then**
    Remove from S any hypothesis inconsistent with d.
    for all $g \in G$ inconsistent with d **do**
      Remove g from G.
      Add to G all minimal specializations h of g such that:
      h is consistent with d, and some member of S is more specific than h.
      Remove from G any hypothesis less general than another in G.
    end for
  end if
end for

Positive examples generalize S (and prune G)l negative examples specialize G (and prune S). The algorithm converges when $S = G = \{h\}$ which means a unique consistent hypothesis) or $S = G = \emptyset$ which means no consistent hypothesis in $mathcal H$.

Picking new training examples An active learner should query for the classification of the instance that best discriminates among remaining hypotheses. Whatever one will shrink version space by up to half.

Classifying new instances with a partially learned concept An instance can be classified confidently once every hypothesis in the version space agrees on it, a vote decides otherwise

Inductive Bias

To fix the fact that constraints can mean the learner cannot represent some concepts, which means Candidate-elimination does not converge:

Fix: an unbiased hypothesis space Let $\mathcal{H}'$ be the power set of $\mathcal{X}$ so every concept is representable.

An unbiased learned cannot generalize beyond the observed examples - this means some bias is needed for inductive generalization.

Consider a concept learning algorithm L for the set of instances $\mathcal{X}$. Let c be an arbitrary target concept,

$$D_c = \{\langle x, c(x)\rangle\}$$

its training data, and $L(x_i, D_c)$ the classification L assigns to a new instance x_i after training on D_c.

The **inductive bias** of L is any minimal set of assertions B such that, for any target concept c and instances D_c,

$$\forall x_i \in \mathcal{X}: \qquad (B \wedge D_c \wedge x_i) \vdash L(x_i, D_c).$$

In short: The bias is the extra assumption that allows deductive inference to reproduce the learner's inductive prediction.

Note: Stronger bias means more generalization from less data, but greater risk of ruling out the true concept if bias is wrong.

Week 2 - Probabilistic Machine Learning

Probabilistic ML: Treats unknown quantities as random variables with probability distributions. Useful for prediction and decision-making under uncertainty.

Supervised Learning

Learn a mapping

$$f : \mathcal{X} \to \mathcal{Y}$$

from inputs x to outputs y using training data

$$D_{\text{train}} = \{(x_n, y_n)\}_{n=1}^{N}.$$

- x: features / covariates / predictors
- y: label / target / response
- **Classification**: y is discrete
- **Regression**: y is continuous

Classification:

$$\mathcal{Y} = \{1, \ldots, C\}$$

where C is the number of mutually exclusive classes. $C = 2$ is binary classification.

Design Matrix: For N examples and D features, $X \in \mathbb{R}^{N \times D}$. Rows = examples; columns = features.

Exploratory Data Analysis (EDA): Inspect data before modeling to identify patterns, relationships, outliers, and data-quality issues.

3.1.1 Empirical Risk Minimization

Misclassification Rate:

$$L(\theta) = \frac{1}{N} \sum_{n=1}^{N} \mathbb{I}[y_n \neq f(x_n; \theta)].$$

Assumes all classification errors have equal cost.

General Loss:

$$\ell(y, \hat{y})$$

measures the cost of predicting $\hat{y}$ when the true output is y.

Empirical Risk:

$$L(\theta) = \frac{1}{N} \sum_{n=1}^{N} \ell(y_n, f(x_n; \theta)).$$

ERM:

$$\boxed{\hat{\theta} = \arg\min_{\theta} L(\theta) = \arg\min_{\theta} \frac{1}{N} \sum_{n=1}^{N} \ell(y_n, f(x_n; \theta))}$$

Goal: Minimize expected loss on unseen data (**generalization**), not merely training loss.

3.1.2 Uncertainty

- **Epistemic uncertainty**: uncertainty from lack of knowledge about the true model; can decrease with more data.
- **Aleatoric uncertainty**: inherent randomness/noise in the data; generally irreducible.

For classification, model a conditional distribution:

$$p(y = c \mid x; \theta) = f_c(x; \theta),$$

where

$$0 \leq f_c(x; \theta) \leq 1, \qquad \sum_{c=1}^{C} f_c(x; \theta) = 1.$$

3.1.3 Softmax Function

A model produces unnormalized scores called **logits**:

$$a = f(x; \theta) \in \mathbb{R}^C.$$

Softmax converts logits into class probabilities:

$$\boxed{\text{softmax}(a)_c = \frac{e^{a_c}}{\sum_{j=1}^{C} e^{a_j}}}$$

Thus,

$$p(y = c \mid x; \theta) = \text{softmax}(f(x; \theta))_c.$$

Properties:

$$0 \leq p(y = c \mid x; \theta) \leq 1, \qquad \sum_{c=1}^{C} p(y = c \mid x; \theta) = 1.$$

Softmax maps

$$\mathbb{R}^C \to [0, 1]^C.$$

3.1.4 Logistic Regression

Special case where the function f is affine,

$$f(x; \theta) = b + w^T x, \text{ with } \theta = (w, b)$$

This is called logistic regression. With w = weights, b = bias. Bias gets absorbed into weights,

$$\hat{w} = [b, w_1, \ldots, w_D], \hat{x} = [1, x_1, \ldots, x_D] \text{so } \hat{w}^T \hat{x} = b + w^T x$$

So affine gets turned into linear.

3.1.5 Maximum Likelihood Estimation

Use negative log probability as the loss,

$$\updownarrow(y, f(x; \theta)) = -\log p(y \mid f(x; \theta))$$

Negative log-likelihood (NLL)

$$NLL(\theta) = -\frac{1}{N} \sum_{n=1}^{N} \log p(y_n \mid f(x_n; \theta))$$

A good model assigns high probability to the true label, which means low NLL

3.1.6 Regression

Definition: Regression

Predict a real-valued output $y \in \mathbb{R}$

Quadratic Loss,

$$\updownarrow_2(y - \hat{y}) = (y - \hat{y})^2$$

Mean Squared Error,

$$MSE(\theta) = \frac{1}{N} \sum_{n=1}^{N} (y_n - f(x_n; \theta))^2$$

Large residuals penalized more (use l_1 instead if outliers are a concern)

3.1.7 Gaussian Regression

Assume the target follows a Gaussian distribution:

$$y \mid x \sim \mathcal{N}(\mu(x), \sigma^2)$$

where $\mu(x) = f(x;\theta)$ is the model prediction.

Gaussian PDF:

$$\boxed{\mathcal{N}(y \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y-\mu)^2}{2\sigma^2}\right)}$$

For fixed σ^2, the negative log-likelihood is

$$\boxed{\text{NLL}(\theta) = \frac{1}{2\sigma^2}\,\text{MSE}(\theta) + \text{const}}$$

Therefore,

$$\boxed{\text{MLE under Gaussian likelihood} \equiv \text{least-squares regression}}$$

Key idea: Maximizing Gaussian likelihood is equivalent to minimizing squared error when variance is fixed.

3.1.8 Linear Regression

Simple Linear Regression (1D):

$$f(x;\theta) = b + wx$$

Multiple Linear Regression:

$$\boxed{f(x;\theta) = b + w_1x_1 + \cdots + w_Dx_D = b + w^Tx}$$

where b is the bias/intercept and w contains the feature weights.

Parameters are chosen to minimize mean squared error:

$$\boxed{\hat{\theta} = \arg\min_{\theta} \text{MSE}(\theta)}$$

Least Squares: Finds the parameters that minimize the sum of squared residuals (differences between predicted and true values).

$$\text{residual}_n = y_n - f(x_n;\theta)$$

Key connection:

$$\text{Linear Regression} + \text{Gaussian Noise} \quad \Longrightarrow \quad \text{Least Squares / MSE}$$

3.1.9 Polynomial Regression & Feature Engineering

Polynomial regression transforms the input into polynomial features:

$$\boxed{f(x;w) = w^T\phi(x), \qquad \phi(x) = [1, x, x^2, \ldots, x^D]^T}$$

- Still **linear in the parameters** w, despite being nonlinear in x.
- $\phi(x)$ is a **feature transformation**.
- Manually choosing $\phi(x)$ is called **feature engineering**.
- **Cross terms**, such as x_1x_2, model interactions between features.

Example 2D quadratic features:

$$\phi(x) = [1, x_1, x_2, x_1^2, x_2^2, x_1x_2]^T.$$

Key idea:

$$\text{nonlinear features} + \text{linear weights} \Rightarrow \text{polynomial regression.}$$

Degrees of Freedom and Overfitting

As polynomial degree $D \to N-1$: one parameter per data point, leads to perfect interpolation MSE $= 0$ on training data. However, model overfits and predicts poorly on new inputs.

Transition to Deep Neural Networks

Learn the feature transform itself: let $\phi(x;V)$ have its own parameters V:

$$f(x;w,V) = w^T\phi(x;V)$$

Recursively compute:

$$f(x;\theta) = f_L(f_{L-1}(\ldots f_1(X)\ldots))$$

Final layer is linear, so the function itself is a learned feature extractor - core idea of deep neural networks & CNNS.

3.1.10 Empirical Risk vs. Population Risk

Empirical Risk: Average loss on the training data.

$$\boxed{L(\theta; D_{\text{train}}) = \frac{1}{|D_{\text{train}}|} \sum_{(x,y)\in D_{\text{train}}} \ell(y, f(x;\theta))}$$

Population Risk: Expected loss over the true data distribution p^*.

$$\boxed{L(\theta; p^*) = \mathbb{E}_{(x,y)\sim p^*}\left[\ell(y, f(x;\theta))\right]}$$

Generalization Gap:

$$\boxed{L(\theta; p^*) - L(\theta; D_{\text{train}})}$$

- Small gap $\Rightarrow$ model generalizes well.
- Large gap with low training risk $\Rightarrow$ **overfitting**.

Approach

Data is split into training, validation, and test sets. Training shouldn't be used in entirely as model would grow too complex and overfit, so we use validation sets to minimize risk, and a final test set to give an unbiased performance estimate.

Unsupervised Learning

Given only inputs $D_{train} = \{x_n\}_{n=1}^N$, fit an unconditional model p(x) that can generate new data, vs supervised learning's conditional model $p(y \mid x)$

3.3.1 Clustering

Partition inpupts into regions of "similar" points

3.3.2 Factor Analysis & Latent Variables

Assume high-dimensional data $x_n \in \mathbb{R}^D$ is generated from lower-dimensional **latent variables** $z_n \in \mathbb{R}^K$, typically $K < D$.

$$z_n \to x_n$$

Factor Analysis (FA):

$$\boxed{p(x_n \mid z_n;\theta) = \mathcal{N}(x_n \mid Wz_n + \mu, \Sigma)}$$

- Similar to linear regression, but z_n is **unobserved**.
- W: maps latent factors to observed features.
- μ: mean; Σ: noise covariance.
- If $\Sigma = \sigma^2 I \Rightarrow$ **Probabilistic PCA**.
- Nonlinear mapping $f(z_n;\theta) \Rightarrow$ basis for **Variational Autoencoders (VAEs)**.

Key idea:

$$\boxed{\text{low-dimensional latent } z \longrightarrow \text{high-dimensional observed } x}$$

3.3.3 Self-supervised Learning

Create proxy supervised tasks from unlabeled data, e.g. predict color from grayscale.

- Learns useful features without needing the "true" latent factors z
- Features transferred to downstream supervised tasks

3.3.4 Evaluating Unsupervised Learning

Unsupervised learning is difficult to evaluate because there is often **no ground truth**.

1. Density Estimation: Evaluate negative log-likelihood (NLL) on held-out data:

$$\boxed{L(\theta; \mathcal{D}) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \log p(x \mid \theta)}$$

- Lower NLL $\Rightarrow$ model assigns higher probability to real data.
- Can be difficult in high dimensions or affected by memorization.

2. Downstream Task Performance: Evaluate whether the learned representation improves performance or **sample efficiency** on a supervised task.

Other Goal: Unsupervised learning may prioritize **interpretability** or scientific understanding rather than predictive performance.

Reinforcement Learning

An agent learns a policy $a = \pi(x)$ mapping states to actions by interacting with an environment and receiving occasional reward signals.

Preprocessing Discrete Inputs

One-Hot Encoding: Convert a K-valued categorical feature into a binary vector:

$$\boxed{\text{onehot}(x) = [\mathbb{I}[x = 1], \ldots, \mathbb{I}[x = K]]}$$

Only the entry corresponding to the category is 1; all others are 0.

Feature Crosses: Add features representing combinations of categorical values to capture **interaction effects**.

$$\mathbb{I}[x_1 = k,\ x_2 = j]$$

- One-hot features capture individual category effects.
- Feature crosses capture interactions between categories.

Preprocessing Text: Bag of Words

Bag of Words (BoW): Represent a document by the number of times each vocabulary word occurs.

$$\boxed{\tilde{x}_{nv} = \sum_{t=1}^{T} \mathbb{I}[x_{nt} = v]}$$

where $\tilde{x}_{nv}$ is the count of word v in document n.

- Converts documents into vectors in $\mathbb{R}^D$.
- **Ignores word order**.
- Common preprocessing: lowercase, remove stop words, stemming.
- Collection of documents forms a **term-frequency matrix**.

TF-IDF & Word Embeddings

TF-IDF: Weights words by importance, downweighting common, uninformative words.

$$\boxed{\text{IDF}_i = \log \frac{N}{1 + \text{DF}_i}}$$

$$\boxed{\text{TFIDF}_{ij} = \log(\text{TF}_{ij} + 1)\, \text{IDF}_i}$$

- TF = frequency of word i in document j.
- DF = number of documents containing word i.
- High TF + low DF $\Rightarrow$ important word.

Word Embeddings: Map sparse one-hot vectors to low-dimensional dense vectors:

$$\boxed{e = Ex, \qquad E \in \mathbb{R}^{K \times V}}$$

Similar words should have nearby embeddings.

Out-of-Vocabulary Words

OOV Problem: Test data may contain words not present in the training vocabulary.

- Simple solution: map unknown words to an **UNK** token.
- Better: split words into **subword units / wordpieces**.
- **Byte-Pair Encoding (BPE)** learns common subword units, allowing unseen words to be represented from known pieces.

Handling Missing Data

Let $M_{nd} = 1$ if feature d of example n is missing.

- **MCAR – Missing Completely At Random:** missingness is independent of all data.

$$p(M \mid X, Y) = p(M)$$

- **MAR – Missing At Random:** missingness depends only on **observed** values.
- **NMAR – Not Missing At Random:** missingness depends on the **missing value itself**; the missingness mechanism must be modeled.

Imputation: Replace missing values with estimates. Simple approach: **mean imputation**; more advanced approaches model the missing values probabilistically.

Week 3 - Probability Specifics

There are two interpretations of probability:

- **Frequentist** - probability - long-run frequency of a repeatable event (Fair coin $\tilde{5}0\%$)
- **Bayesian** - probability - degree of belief / uncertainty, even for one-off events. **Focus of machine learning**

Definition: Event

A binary proposition A that is true or false.

$$0 \leq \Pr(A) \leq 1, \qquad \Pr(\bar{A}) = 1 - \Pr(A)$$

$$\Pr(A, B) = \Pr(A) \Pr(B) \qquad \text{if } A, B \text{ independent}$$

$$\Pr(A \vee B) = \Pr(A) + \Pr(B) - \Pr(A, B), \qquad = \Pr(A) + \Pr(B) \quad \text{if mutually exclusive}$$

Week 3 - Probability Theory

$$\Pr(B \mid A) \triangleq \frac{\Pr(A, B)}{\Pr(A)} \qquad (\text{undefined if } \Pr(A) = 0)$$

$$A \text{ independent of } B \iff \Pr(A, B) = \Pr(A) \Pr(B)$$

$$A \perp B \mid C \iff \Pr(A, B \mid C) = \Pr(A \mid C) \Pr(B \mid C)$$

Intuition:

Variables are rarely *unconditionally* independent, but conditioning on the right intermediate variable C often renders them (conditionally) independent: graph $A - C - B$.

Definition: Random variable (r.v)

An unknown variable quantity X; its set of possible values X is the *sample space* / state space. An **event** is a subset of outcomes.

Probability Mass Function (pmf) - discrete

$$\Pr(x) = \Pr(X = x), 0 \leq \Pr(x) \leq 1, \sum_{x \in X} \Pr(x) = 1$$

Cumulative Distribution Function:

$$P(x) = \Pr(X \leq x), \Pr(a < X \leq b) = P(b) - P(a)$$

Probability Density Function:

$$\Pr(x) = \frac{d}{dx} P(x)$$

This gives us,

$$\Pr(a < X \leq b) = \int_a^b p(x)dx, \Pr(x < X \leq x + dx) \approx p(x)dx$$

Quantiles

$$F^{-1}(q) = x_q, \quad P(X \leq x_q) = q$$

Median: $q = 0.5$; Quartiles: $q = 0.25, 0.75$.

For $X \sim \mathcal{N}(\mu, \sigma^2)$:

$$P(\mu - 1.96\sigma \leq X \leq \mu + 1.96\sigma) \approx 0.95$$

5.0.1 Joint, Marginal, Conditional

$$p(x) = \sum_y p(x, y), \qquad p(x, y) = p(x)p(y \mid x)$$

$$p(x_{1:D}) = \prod_{i=1}^{D} p(x_i \mid x_{1:i-1})$$

5.0.2 Independence

$$X \perp Y \iff p(x, y) = p(x)p(y)$$
$$X \perp Y \mid Z \iff p(x, y \mid z) = p(x \mid z)p(y \mid z)$$

5.0.3 Mean and Variance

$$\mathbb{E}[X] = \sum_x x p(x) \quad \text{or} \quad \int x p(x)\, dx$$

$$\mathrm{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$$
$$\mathbb{E}[aX + b] = a\mathbb{E}[X] + b$$
$$\mathrm{Var}(aX + b) = a^2 \mathrm{Var}(X)$$
$$\mathbb{E}\left[\sum_i X_i\right] = \sum_i \mathbb{E}[X_i]$$

For independent X_i:

$$\mathrm{Var}\left(\sum_i X_i\right) = \sum_i \mathrm{Var}(X_i)$$
$$\mathbb{E}\left[\prod_i X_i\right] = \prod_i \mathbb{E}[X_i]$$

5.0.4 Mode and Conditional Moments

$$x^* = \arg\max_x p(x)$$

Law of total expectation:

$$\mathbb{E}[X] = \mathbb{E}_Y[\mathbb{E}[X \mid Y]]$$

Law of total variance:

$$\mathrm{Var}(X) = \mathbb{E}_Y[\mathrm{Var}(X \mid Y)] + \mathrm{Var}_Y(\mathbb{E}[X \mid Y])$$

5.0.5 Bayes' Rule

$$p(h \mid y) = \frac{p(y \mid h)p(h)}{p(y)}$$
$$p(y) = \sum_h p(y \mid h)p(h)$$
$$\underbrace{p(h \mid y)}_{\text{posterior}} \propto \underbrace{p(y \mid h)}_{\text{likelihood}} \underbrace{p(h)}_{\text{prior}}$$

5.0.6 Bernoulli Distribution:

$$Y \sim \mathrm{Ber}(\theta): \quad \mathrm{Ber}(y \mid \theta) = \theta^y (1 - \theta)^{1-y}, \quad \Pr(Y = 1) = \theta$$

where θ denotes probability of success

5.0.7 Binomial Distribution:

E.g. number of heads in N Bernoulli trials (s represents # successes)

$$\mathrm{Bin}(s \mid N, \theta) = \binom{N}{s} \theta^s (1 - \theta)^{N-s}, \binom{N}{k} = \frac{N!}{(N-k)!k!}$$

5.0.8 Sigmoid (Logistic) Function:

$$p(y \mid x, \theta) = \mathrm{Ber}(y \mid \sigma(f(x; \theta)))\sigma(a) = \frac{1}{1 + e^{-a}} \quad maps\ \mathbb{R} \to [0, 1]$$

5.0.9 Binary Logistic Regression:

$$p(y \mid x; \theta) = \mathrm{Ber}(\mathrm{y} \mid \sigma(\mathrm{w}^{\mathrm{T}}\mathrm{x} + \mathrm{b})) = \frac{1}{1 + \mathrm{e}^{-(\mathrm{w}^{\mathrm{T}}\mathrm{x} + \mathrm{b})}}$$

One should not use linear regression for classification.

5.0.10 Categorical & Multinomial Distributions:

Categorical

$$\mathrm{Cat}(y \mid \theta) = \prod_{c=1}^{C} \theta_c^{\mathbb{I}[y=c]} = \prod_{c=1}^{C} \theta_c^{y_c}, \sum_c \theta_c = 1$$

Multinomial

$$M(y \mid N, \theta) = \binom{N}{N_1 \ldots N_C} \prod_{c=1}^{C} \theta_c^{N_c}$$

5.0.11 Softmax

$$p(y \mid x, \theta) = \mathrm{Cat}(y \mid f(x; \theta))\, \mathrm{softmax}(a) = [\frac{e^{a_1}}{\sum_{c'} e^{a_{c'}}} \cdots \frac{e^{a_C}}{\sum_{c'} e^{a_{c'}}}]$$

5.0.12 Multiclass Logistic Regression:

$$p(y \mid x; \theta) = \mathrm{Cat}(y \mid \mathrm{softmax}(Wx + b)), p(y = c|x; \theta) = \frac{e^{a_c}}{\sum_{c'} e^{a_{c'}}}$$

5.0.13 Log-Sum-Exp Trick

Numerical stability for softmax (avoids overflow). Let $m = \max_c a_c$:

$$\mathrm{LSE}(a) = \log \sum_c e^{a_c} = m + \log \sum_c e^{a_c - m}$$
$$p(y = c \mid x) = \exp(a_c - \mathrm{LSE}(a))$$

Cross-entropy loss from logits:

$$\mathcal{L} = -\log p(y \mid x) = \mathrm{LSE}(a) - a_y$$

5.0.14 Gaussian CDF:

$$\phi(y; \mu, \sigma^2) = \int_{-\infty}^{y} N(z \mid \mu, \sigma^2) dz = \frac{1}{2}[1 + \mathrm{erf}(\frac{z}{\sqrt{2}})], \ z = \frac{y - \mu}{\sigma}$$

5.0.15 Gaussian PDF:

$$N(y \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\big(-\frac{1}{2\sigma^2}(y - \mu)^2\big)$$

5.0.16 Gaussian Regression

$$p(y \mid x; \theta) = \mathcal{N}(y \mid \mu(x), \sigma^2(x))$$

Homoscedastic: Constant variance.

$$y \mid x \sim \mathcal{N}(w^T x + b, \sigma^2)$$

Heteroscedastic: Input-dependent variance.

$$y \mid x \sim \mathcal{N}(\mu(x), \sigma^2(x))$$

Softplus ensures $\sigma^2(x) > 0$.

95% predictive interval:

$$[\mu(x) - 1.96\sigma(x),\ \mu(x) + 1.96\sigma(x)]$$

5.0.17 Why is Gaussian so widely used?

- 2 easily interpreted parameters (mean,variance)
- **Central Limit Theorem**: sums of iid rv's → Gaussian - great for modeling noise
- **Maximum Entropy**: fewest assumptions given a fixed mean/variance
- Simple,closed form math → tractable algorithms

5.0.18 Dirac Delta & Truncated Gaussian

Dirac delta: Point mass at μ.

$$\lim_{\sigma \to 0} \mathcal{N}(x \mid \mu, \sigma^2) = \delta(x - \mu)$$

$$\int_{-\infty}^{\infty} \delta(x)\, dx = 1, \qquad \int f(y)\delta(x - y)\, dy = f(x)$$

Truncated Gaussian: Normal restricted to (a, b).

$$p(x) = \frac{\frac{1}{\sigma}\phi(\frac{x-\mu}{\sigma})}{\Phi(\frac{b-\mu}{\sigma}) - \Phi(\frac{a-\mu}{\sigma})} \mathbf{1}\{a < x < b\}$$

ϕ: standard normal PDF; Φ: standard normal CDF.

5.0.19 Empirical Distribution

Given N samples $\{x^{(1)}, \ldots, x^{(N)}\}$:

$$\hat{p}_N(x) = \frac{1}{N} \sum_{n=1}^{N} \delta(x - x^{(n)})$$

$$\hat{F}_N(x) = \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}\{x^{(n)} \leq x\}$$

Empirical CDF: step function with jumps of $1/N$.

5.0.20 Transformations

For $Y = f(X)$, discrete:

$$p_Y(y) = \sum_{x : f(x) = y} p_X(x)$$

Continuous:

$$F_Y(y) = P(f(X) \leq y), \quad p_Y(y) = \frac{d}{dy} F_Y(y)$$

5.0.21 Change of Variables

For invertible $y = f(x)$, $x = g(y)$:

$$p_Y(y) = p_X(g(y)) \left| \frac{dg(y)}{dy} \right|$$

Multivariate:

$$p_Y(y) = p_X(g(y)) |\det J_g(y)|$$

where J_g is the Jacobian of the inverse. Polar coordinates: $|\det J_g| = r$.

5.0.22 Linear Transformations

For $Y = AX + b$, with mean μ and covariance Σ:

$$\mathbb{E}[Y] = A\mu + b, \qquad \text{Cov}(Y) = A\Sigma A^T$$

Scalar $Y = a^T X + b$:

$$\text{Var}(Y) = a^T \Sigma a$$

$$\text{Var}(X_1 + X_2) = \Sigma_{11} + \Sigma_{22} + 2\Sigma_{12}$$

5.0.23 Convolution Theorem

For independent X_1, X_2, $Y = X_1 + X_2$:

$$p_Y(y) = (p_1 * p_2)(y) = \int p_1(x) p_2(y - x)\, dx$$

Sum of independent Gaussians:

$$\mathcal{N}(\mu_1, \sigma_1^2) * \mathcal{N}(\mu_2, \sigma_2^2) = \mathcal{N}(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$$

5.0.24 Central Limit Theorem

For iid X_i with mean μ and variance $0 < \sigma^2 < \infty$:

$$S_N = \sum_{i=1}^{N} X_i, \qquad \bar{X} = \frac{S_N}{N}$$

$$Z_N = \frac{S_N - N\mu}{\sigma\sqrt{N}} = \frac{\bar{X} - \mu}{\sigma/\sqrt{N}} \xrightarrow{d} \mathcal{N}(0, 1)$$

For large N:

$$\bar{X} \approx \mathcal{N}\left(\mu, \frac{\sigma^2}{N}\right)$$

5.0.25 Monte Carlo Approximation

Draw $x^{(s)} \sim p_X(x)$ and compute $y^{(s)} = f(x^{(s)})$.

$$\hat{p}_S(y) = \frac{1}{S} \sum_{s=1}^{S} \delta(y - y^{(s)})$$

Monte Carlo expectation:

$$\mathbb{E}[f(X)] \approx \frac{1}{S} \sum_{s=1}^{S} f(x^{(s)})$$

Week 3 - Training Regression Models

Common Notation:

- Divide data into training, validation, and test sets
- m: size (number of instances)
- $x^{(i)}$ is th vector of all feature values of the i-th instance, and $y^{(i)}$ is its label (desired output value)
- X is the data which contains the feature rows $(x^{(i)})^T$
- h is the prediction function / hypothesis.

$$\hat{y}^{(i)} = h(x^{(1)}$$

- Root Mean Squared Error -

$$RMSE(X.h) = \sqrt{\frac{1}{m} \sum_{i=1}^{m} (h(x^{(i)}) - y^{(i)})^2}$$

  where

$$\sqrt{(h(x^{(i)}) - y^{(i)})^2}$$

  is the *Euclidean norm* or l_2 norm, $|| \cdot ||_2$
- Mean Absolute Error is defined as -

$$MAE(X.h) = \frac{1}{m} \sum_{i=1}^{m} ||h(x^{(i)}) - y^{(i)}||_1$$

  where $|| \cdot ||_1$ is the l_1 norm

Linear Regression

Prediction model with n features:

$$\hat{y} = w_0 + \sum_{j=1}^{n} w_j x_j = \mathbf{w}^T \mathbf{x}$$

where $x_0 = 1$ incorporates the bias w_0.

Mean Squared Error (MSE):

$$\text{MSE}(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^{m} (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)})^2$$

Vectorized MSE:

$$\text{MSE}(\mathbf{w}) = \frac{1}{m} \|X\mathbf{w} - \mathbf{y}\|_2^2$$

Closed-form solution (Normal Equation):

$$\boxed{\mathbf{w}^* = (X^T X)^{-1} X^T \mathbf{y}}$$

Requires $X^T X$ to be invertible. Otherwise use the pseudoinverse:

$$\mathbf{w}^* = X^+ \mathbf{y}$$

6.2.1 Computational Complexity

Normal Equation computes the inverse of X which has a complexity of $O(n^{2.4}) \sim O(n^3)$ depending on implementation. This becomes **slow** with more **features**. However, it can handle more instances fine.

Gradient Descent:

A generic optimization algorithm. General Idea

- Iteratively tweak parameters to minimize a cost function
- Measure local gradient of the error function with regards to the parameter (weight) vector w
- Descend in the direction of the gradient until gradient is zero - reached a minimum

1. Randomly initialize weight vector w
2. Decrease the cost function with each iteration until convergence to minimum or early termination

Hyperparameter: Learning rate - η - Determines the size each step will take.

- If η is too small, algorithm will take a lot of iterations and time to converge
- If η is to large, then the algorithm will take fewer iterations, but may miss true solution

Gradient Descent works with quadratic (convex) cost/loss functions - because they have a "bottom".

Problems with Gradient Descent:

- Not all cost functions are convex, perfectly "bowl" shaped. Means it may converge to local minimum not global minimum
- Needs standardization

Batch Gradient Descent

Gradient of MSE:

$$\nabla_{\mathbf{w}} \text{MSE} = \frac{2}{m} X^T (X\mathbf{w} - \mathbf{y})$$

Note this formula involves calculations over the full training set X at each step. Hence why it is called Batch Gradient Descent. As a result, it is **terribly slow** on very large training sets. However, it scales well with number of features. **Opposite of Normal Equation**

Gradient Descent Update:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \text{MSE}(\mathbf{w})$$

where η is the learning rate.

Early Stop

A good technique is to set the number of iterations to very high, and terminate when the gradient becomes tiny, smaller than a tolerance ϵ.

Stochastic Gradient Descent

Instead of using whole training set, pick a random (stochastic) instance in the training set at every step and compute the gradient.

Results

- Much Faster
- Less Regular, bounces/oscillates around the minimum
- If cost function is very irregular, can jump out of local minima, making it better than Batch Gradient Descent. However, **it still never settles at true minimum**

Learning Schedule

A trick to combat the randomness, reduce step size (learning rate) with the number of epochs. The function that determines how and when to reduce the learning is called a *Learning schedule.*

Mini-batch Gradient Descent

Combines Batch and Stochastic Gradient Descent. At each step, compute the gradients on *small random sets of instances called minibatches.*

- Can get performance boost from hardware optimization of matrix operations
- Less erratic then SGD, but may not escape local minima

Table 1: Comparison of algs. for Linear Regression

Alg.	Large m	Large n	Hyperparam.	Scaling?
Normal Eq	fast	slow	0	No
Batch GD	slow	fast	2	yes
Stochastic GD	fast	fast	≥ 2	yes
Minbatch GD	fast	fast	≥ 2	yes

Polynomial Regression

Useful for representing nonlinear data. Fit a linear model by

- add powers of each feature as a new feature
- train a linear model on these new features

Important to note

The new features may be a combinatorial result of the existing features, this means a lot of features which will make the Normal Equation very slow.

Overfitting vs Underfitting

Use learning curves, which plot training set size / number of epochs vs training accuracy and validation accuracy.

- If training accuracy is way higher than validation accuracy $\rightarrow$ overfitting.
- If both training and validation are similar and plateauing $\rightarrow$ underfitting.

Bias vs Variance

Bias - High bias model contributes to under-fitting the training data

Variance - A model with many degrees of freedom is likely to have high variance, thus overfitting the training data

$$\text{complexity} \implies \text{variance increase and bias decreases}$$

Irreducible Error - Due to noisiness of the data itself, needs better preprocessing or data selection.

Regularization

Tools used to combat overfitting, higher-variance models.

6.13.1 Ridge Regression (L2)

Penalizes large weights to reduce overfitting.

$$J(w) = \text{MSE}(w) + \alpha \sum_{i=1}^{n} w_i^2$$

α: regularization strength. Larger α increases bias, reduces variance. Bias term w_0 is typically not penalized.

Closed-form solution:

$$\hat{w} = (X^T X + \lambda D)^{-1} X^T y$$

where $D = \text{diag}(0, 1, \ldots, 1)$ excludes the bias, and $\lambda = m\alpha$ for MSE defined with factor $1/m$.

6.13.2 Lasso Regression (L1)

Penalizes absolute weights; encourages sparsity and automatic feature selection.

$$J(w) = \text{MSE}(w) + \alpha \sum_{i=1}^{n} |w_i|$$

α: regularization strength. Larger α encourages more zero weights. Bias w_0 is typically not penalized.

Subgradient of L1 penalty:

$$\partial |w_i| = \begin{cases} -1 & w_i < 0 \\ [-1, 1] & w_i = 0 \\ 1 & w_i > 0 \end{cases}$$

$$g_i = \frac{\partial \,\text{MSE}}{\partial w_i} + \alpha \,\text{sign}(w_i), \quad w_i \neq 0$$

At $w_i = 0$, use a valid subgradient.

Ridge vs. Lasso:

- Ridge (L2): Shrinks weights toward zero.
- Lasso (L1): Can set weights exactly to zero.

6.13.3 Elastic Net Regression (L1 + L2)

Combines Ridge and Lasso regularization.

$$J(w) = \text{MSE}(w) + r\alpha \sum_{i=1}^{n} |w_i| + \frac{(1-r)\alpha}{2} \sum_{i=1}^{n} w_i^2$$

$r \in [0, 1]$: L1/L2 mixing ratio; α: regularization strength.

- $r = 0$: Ridge (L2).
- $r = 1$: Lasso (L1).
- $0 < r < 1$: Elastic Net.
- Prefer Lasso/Elastic Net for sparse features.
- Prefer Elastic Net for correlated features or when $n > m$ (features > samples).

Logistic Regression

Binary classification; estimates $P(y = 1 \mid x)$.

Sigmoid function:

$$\hat{p} = \sigma(w^T x), \qquad \sigma(t) = \frac{1}{1 + e^{-t}}$$

Classification threshold:

$$\hat{y} = \begin{cases} 1 & \hat{p} \geq 0.5 \\ 0 & \hat{p} < 0.5 \end{cases}$$

Equivalently, $\hat{y} = 1$ iff $w^T x \geq 0$.

Binary Cross-Entropy (BCE):

$$c(w) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p})$$

Average loss:

$$J(w) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log \hat{p}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

Gradient:

$$\nabla_w J(w) = \frac{1}{m} X^T (\hat{p} - y)$$

Component-wise:

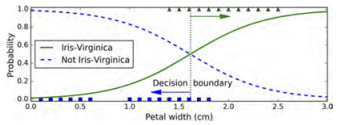
$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} (\hat{p}^{(i)} - y^{(i)}) x_j^{(i)}$$

Gradient Descent:

$$w \leftarrow w - \eta \nabla_w J(w)$$

No general closed-form solution; BCE is convex in w for standard logistic regression. Supports Batch, SGD, and Mini-Batch GD.

Example Decision Boundary



Softmax Regression

Multiclass extension of Logistic Regression (K classes).

Class scores (logits):

$$s_k(x) = w_k^T x, \qquad s(x) = W^T x$$

Softmax probabilities:

$$\hat{p}_k = P(y = k \mid x) = \frac{e^{s_k(x)}}{\sum_{j=1}^{K} e^{s_j(x)}}$$

$$\sum_{k=1}^{K} \hat{p}_k = 1$$

Classification:

$$\hat{y} = \arg\max_k \hat{p}_k = \arg\max_k s_k(x)$$

Cross-Entropy Loss:

$$J(W) = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} y_k^{(i)} \log \hat{p}_k^{(i)}$$

$y_k^{(i)} \in \{0, 1\}$: one-hot encoded labels.

Gradient for class k:

$$\nabla_{w_k} J(W) = \frac{1}{m} \sum_{i=1}^{m} (\hat{p}_k^{(i)} - y_k^{(i)}) x^{(i)}$$

Vectorized Gradient:

$$\nabla_W J(W) = \frac{1}{m} X^T (P - Y)$$

P: predicted probabilities; Y: one-hot labels.

Gradient Descent:

$$W \leftarrow W - \eta \nabla_W J(W)$$

Week 4 - Decision Trees

Decision tree representation

- Each internal node tests an attribute
- Each branch corresponds to an attribute value
- Each leaf node assigns a classification

When to consider Decision Trees

- Instances describable by attribute-value pairs
- Target function is discrete valued
- Disjunctive (mutually exclusive) hypothesis may be required
- Possibly noisy training data

Entropy

Consider S to be the sample of training examples. Entropy measures the impurity of S.

$$\text{Entropy}(S) = -p \log_2 p - p \log_2 p$$

Entropy(S) is the expected number of bits needed to encode a class (negative or positive) of randomly drawn member of S. Gain(S, A) =expected reduction in entropy due to sorting on A.

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

ID3 Algorithm - Decision Trees

ID3 (Iterative Dichotomiser 3) builds a decision tree using a greedy, top-down approach, selecting the attribute with the highest **Information Gain** at each node.

Algorithm:

1. Start with the entire training dataset.
2. Calculate entropy of the current dataset.
3. Compute Information Gain for each available attribute.
4. Select the attribute with the highest Information Gain.
5. Split the dataset based on the selected attribute's values.
6. Recursively repeat until a stopping condition is met.

Entropy: Measures uncertainty or impurity.

$$H(S) = -\sum_{i=1}^{k} p_i \log_2(p_i)$$

Information Gain: Reduction in entropy after splitting on attribute A.

$$IG(S, A) = H(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} H(S_v)$$

Stopping Conditions:

- All samples belong to one class → create leaf.
- No attributes remain → predict majority class.
- Empty subset → use parent's majority class.

Advantages:

- Simple and interpretable.
- No feature scaling required.
- Automatically selects relevant features.
- Robust to noisy data.

Disadvantages:

- Prone to overfitting.
- Biased toward attributes with many distinct values.
- Standard ID3 handles categorical features.

Key Takeaway: ID3 greedily maximizes Information Gain (minimizes weighted entropy) at each split, but does not guarantee a globally optimal tree.

Occam's Razor for Decision Tree

Why do we prefer short hypotheses?

Pros:

- fewer short hypotheses than long hypotheses
- short hypothesis that fits data unlikely to be coincidence, while long one might be

Cons

- Many ways to define small sets of hypotheses

Overfitting in Decision Trees

A hypothesis $h \in H$ **overfits** the training data if there exists an alternative hypothesis $h' \in H$ such that:

$$error_{train}(h) < error_{train}(h')$$
and
$$error_{\mathcal{D}}(h) > error_{\mathcal{D}}(h')$$

Where:

- $error_{train}(h)$: Error on the training dataset.
- $error_{\mathcal{D}}(h)$: Error over the true data distribution.
- h': Higher training error but lower generalization error.

Key Takeaway: Overfitting occurs when a model learns training-specific patterns or noise, resulting in low training error but poor generalization to unseen data.

Avoiding Overfitting:

Stop growing tree when splits become not statistically significant. Or *prune* tree after its fully grown.

Selecting the best (most general) trees

Measure performance over training data, then over separate validation data set.

Use **MDL - Minimum Description Length**:

$$\text{MIN}_{tree}\{size(tree) + size(misclassifications(tree))\}$$

Rule Post-Pruning

Rule Post-Pruning reduces overfitting by converting a decision tree into IF-THEN rules and pruning each rule independently.

Algorithm:

1. Train a fully grown decision tree.
2. Convert each root-to-leaf path into an IF-THEN rule.
3. Remove conditions that do not improve estimated generalization performance.
4. Evaluate pruned rules using validation data or a pruning metric.
5. Order rules by estimated accuracy.

Example:

$$(A > 5) \wedge (B = 1) \wedge (C < 10) \implies Y = 1$$
$$\Downarrow$$
$$(A > 5) \wedge (B = 1) \implies Y = 1$$

Advantages:

- Each rule can be simplified independently.
- More flexible than pruning entire subtrees.
- Produces interpretable classification rules.

Key Takeaway: Simplify individual decision rules rather than pruning entire subtrees.

Attributes with Many Values

The issue is that if an attribute has many values, *Gain* will select it. Fix: use *GainRatio*

$$GainRatio(S, A) = \frac{Gain(S, A)}{SplitInformation(S, A)} \; SplitInformation(S, A) = -\sum_{i=1}^{c} \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$

Attributes with Costs

Some attributes have an associated cost (e.g., medical tests, computation time). The goal is to build a with low expected classification cost.

Instead of maximizing Information Gain alone, use cost-sensitive splitting criteria:

$$\frac{Gain^2(S, A)}{Cost(A)}$$

Or,

$$\frac{2^{Gain(S,A)} - 1}{(Cost(A) + 1)^w}$$

Where $w \in [0, 1]$ controls the importance of cost.

- $w = 0$: Ignore attribute cost.
- $w = 1$: Apply full cost penalty.
- Higher cost reduces an attribute's splitting preference.

Key Takeaway: Prefer attributes that provide high Information Gain relative to their evaluation cost.

Unknown Attribute Values

Missing attribute values can be handled during both training and classification.

Approaches:

1. Replace missing values with the most common value of attribute A among examples at the node.
2. Replace with the most common value of A among examples with the same target class.
3. Assign probabilities p_i to possible values v_i of attribute A.
   - Distribute fraction p_i of the example to each corresponding subtree.

For new examples with missing attributes, use the same strategies during classification.

Key Takeaway: Handle missing values using imputation or probabilistic distribution across branches.

CART - Classification and Regression Trees

CART (Classification and Regression Trees)

CART builds binary decision trees for both classification and regression using greedy recursive partitioning.

Numeric splits: Test $x_k \le t_k$ (left), otherwise right. Candidate thresholds are midpoints of consecutive sorted feature values. At each node CART chooses the locally best split; it does not plan ahead, so the resulting tree need not be globally optimal.

Algorithm:

1. Start with the entire training dataset.
2. Evaluate possible binary splits for each feature.
3. Select the split minimizing weighted impurity (classification) or prediction error (regression).
4. Recursively split until a stopping condition is met.
5. Optionally prune the tree to prevent overfitting.

Classification: Typically uses Gini Impurity.

$$Gini(S) = 1 - \sum_{i=1}^{k} p_i^2$$

Choose the split minimizing weighted Gini:

$$Gini_{split} = \frac{|S_L|}{|S|} Gini(S_L) + \frac{|S_R|}{|S|} Gini(S_R)$$

Impurity reduction:

$$\Delta G = G(S) - Gini_{split} \ge 0.$$

For K classes, $0 \le G \le 1 - 1/K$; $G = 0$ iff the node is pure, and the maximum occurs for equally frequent classes. Equivalently, $G = \sum_k p_k(1 - p_k)$. Misclassification impurity is $1 - \max_k p_k$, but Gini/entropy are more sensitive to changes in class proportions when selecting splits.

Regression: Typically minimizes squared error.

$$SSE = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Each regression leaf predicts the mean target value of its training samples.

Key Properties:

- Always produces binary splits.
- Supports numerical and categorical attributes.
- Handles classification and regression.
- Uses cost-complexity pruning to reduce overfitting.

Stopping: Stop if node is pure, no split improves impurity, max_depth is reached, or $min_samples_split$ is unmet. Leaf class prediction is the majority class. **Balanced-tree complexity:** Prediction $O(\log m)$; training approximately $O(nm \log m)$ for m samples and n features (under the slide's efficient split-search assumptions). **Iris root example:** $G = 1 - 3(50/150)^2 = 2/3$. Splitting 50 pure Setosa from 100 mixed others gives $J = (50/150)(0) + (100/150)(0.5) = 1/3$, so $\Delta G = 1/3$.

Cost-Complexity Pruning:

$$R_\alpha(T) = R(T) + \alpha|T|$$

where $R(T)$ is the tree's total leaf error or impurity, $|T|$ is the number of leaves, and $\alpha \ge 0$ controls the complexity penalty.

Pre-pruning: Stop growth early through depth/sample limits. **Post-pruning:** Grow then remove subtrees that do not justify added complexity. Select regularization strength (e.g. α) using cross-validation; larger α favors fewer leaves.

Key Takeaway: CART greedily constructs binary trees using Gini impurity for classification or squared error for regression, then optionally prunes the tree.

Gini vs Entropy

End trees are quite similar, gini is slightly faster to compute. Gini tends to isolate most frequent class, while entropy tends to produce slightly more balanced trees.

Estimating class probabilities with decision trees

Traverse the tree to find the lead node for instance x. Return the ration of training instances of class k in this node.

Let $\ell(x)$ be the leaf reached by x. Then

$$\hat{P}(y = k \mid x) = \frac{\#\{i \in \ell(x) : y_i = k\}}{m_{\ell(x)}}, \qquad \hat{y} = \arg\max_k \hat{P}(y = k \mid x).$$

Every point in the same leaf receives identical class probabilities; tree probabilities and predictions are **piecewise constant**. Iris example: leaf counts $[0, 49, 5]$ give probabilities $[0, 49/54, 5/54]$.

Regularization: Hyperparameters

- max_depth
- min_sample_split The minimum number of examples a node must have before it can be split
- $min_samples_leaf$ The minimum number of samples a leaf node must have
- $min_weight_fraction_leaf$ This is the same as $min_samples_leaf$, but expressed as a fraction of the total number of weighted instances
- max_leaf_nodes maximum number of leaf nodes
- $max_features$ maximum number of features that are evaluated for splitting at each node

A tree is **nonparametric**: its size is not fixed beforehand. Increasing min_* or decreasing max_* parameters restricts complexity. Greater depth can improve training accuracy while harming test accuracy; choose settings using validation or cross-validation.

Regression Tree

At each node compute node value, and MSE. Node value is obtained by averaging the values of the nodes in that node.

Regression leaves produce **piecewise-constant** predictions (a step function in one dimension). More splits give finer steps, but unrestricted trees can fit noise; $min_samples_leaf$ limits overfitting in regression as well.

7.15.1 CART Cost Function for Regression

CART selects the binary split that minimizes the weighted Mean Squared Error (MSE) of the resulting subsets.

$$J(k, t_k) = \frac{m_{left}}{m} MSE_{left} + \frac{m_{right}}{m} MSE_{right}$$

Where:

- k: Feature selected for splitting.
- t_k: Threshold for feature k.
- m: Total number of instances at the node.
- $m_{left/right}$: Number of instances in each subset.
- $MSE_{left/right}$: Mean squared error of each subset.

Mean Squared Error:

$$MSE_{node} = \frac{1}{m_{node}} \sum_{i \in node} (y_i - \hat{y}_{node})^2$$

Leaf Prediction: Mean target value of the node.

$$\hat{y}_{node} = \frac{1}{m_{node}} \sum_{i \in node} y_i$$

Optimal Split:

$$(k^*, t_k^*) = \arg\min_{k, t_k} J(k, t_k)$$

Key Takeaway: Regression CART minimizes weighted child MSE, favoring splits that group similar target values together.

Since $\hat{y}_{node}$ is the within-node mean, it minimizes SSE. The split's total within-child SSE is $mJ(k, t_k)$; choosing the smallest J maximizes the reduction in squared error.

Limitations of Decision Trees

Instability to rotation, due to having orthogonal decision boundaries. One way to limit this problem is to use Principal Component Analysis. **Instability to details in the data**

A one-feature threshold creates an axis-parallel decision boundary; rotation can turn a simple boundary into many staircase splits. PCA may align informative directions with the axes. A small training-data change or an arbitrary tie-break at the root can change entire subtrees. Hence individual trees have **high variance**, motivating bagging and Random Forests.

Week 4 - Entropy and Information Gain

Entropy

Entropy measures uncertainty or information content in bits.

$$H(X) = -\sum_{i=1}^{n} p_i \log_2 p_i$$

- $H(X) \geq 0$; $H(X) = 0$ when one outcome is certain.
- Maximum entropy occurs for a uniform distribution: $H(X) = \log_2 n$.
- Fair coin: $H(0.5, 0.5) = 1$ bit.
- Binary entropy:
$$H(p, 1-p) = -p \log_2 p - (1-p) \log_2 (1-p)$$
- Convention: $0 \log_2 0 = 0$.

Information Gain

Information Gain measures the reduction in entropy after testing an attribute.

$$IG(S, A) = H(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} H(S_v)$$

- S: Training examples.
- S_v: Subset where attribute A has value v.
- Higher Information Gain means a more effective split.
- Decision trees select attributes maximizing IG.

Entropy Properties

Joint Entropy:

$$H(X, Y) = -\sum_{x,y} p(x, y) \log_2 p(x, y)$$

If X, Y are independent:

$$H(X, Y) = H(X) + H(Y)$$

Recursive Property:

$$H(p_1, \ldots, p_n) = H(p_1, 1-p_1) + (1-p_1) H\left(\frac{p_2}{1-p_1}, \ldots, \frac{p_n}{1-p_1}\right)$$

Relative Entropy (KL Divergence):

$$D_{KL}(P \| Q) = \sum_x P(x) \log_2 \frac{P(x)}{Q(x)}$$

Gibbs' Inequality:

$$D_{KL}(P \| Q) \geq 0$$

Equality iff $P = Q$.

Jensen's Inequality: For convex f,

$$f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)]$$

Key Takeaway: Entropy measures uncertainty; Information Gain measures how much an attribute reduces it.

Week 5/6 - Principal Component Analysis

Used for dimensionality reduction, PCA finds the new orthonormal axes, the principal components, along the direction of largest variance. It minimizes the mean squared distance from points to their projections.

1. **Center the data** ($X \in \mathbb{R}^{m \times n}$, rows $x^{(i)}$):

$$\tilde{X} = X - \mathbf{1}\bar{x}^\top, \qquad C = \frac{1}{m-1} \tilde{X}^\top \tilde{X}.$$

2. **Maximize the projected variance:**

$$\max_{\mathbf{w}} \mathbf{w}^\top C \mathbf{w} \quad \text{s.t.} \quad \|\mathbf{w}\|_2 = 1.$$

Lagrangian:

$$\mathcal{L} = \mathbf{w}^\top C \mathbf{w} - \lambda(\mathbf{w}^\top \mathbf{w} - 1).$$

Setting $\nabla_{\mathbf{w}} \mathcal{L} = 0$ gives:

$$C\mathbf{w} = \lambda \mathbf{w} \quad \Rightarrow \quad \mathbf{w}^\top C \mathbf{w} = \lambda.$$

- The principal components are the eigenvectors $\mathbf{w}_1, \ldots, \mathbf{w}_n$ of C, ordered by eigenvalue:
$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n \geq 0.$$
- λ_j is the variance of the data along $\mathbf{w}_j$.

- In practice, use SVD:

$$\tilde{X} = U\Sigma V^\top.$$

The columns of V are the principal components, and

$$\lambda_j = \frac{\sigma_j^2}{m-1}.$$

3. **Project onto the first d principal components:**

$$Z = \tilde{X}W_d, \qquad W_d = [\mathbf{w}_1 \ \cdots \ \mathbf{w}_d].$$

Notes:

- To keep x% of the data, sum the principal components and figure out the combinations to reach it. For example,

$$\text{EVR}_j = \frac{\lambda_j}{\sum_{i=1}^n \lambda_i}, \qquad d^* = \min\left\{d : \sum_{j=1}^{d} \text{EVR}_j \geq 0.95\right\}$$

- Standardize the features first when their units differ
- PCA is unsupervised, it ignore labels
- Can be reconstructed using

$$\hat{X} = ZW_d^T + \mathbf{1}\bar{x}^T$$

Week 5/6 - More CART Tree stuff

Strength and Weaknesses of Decision Trees

Strengths

- Interpretability(white-box)
- Little data preparation
- handles numerical and categorical features (multi-class)
- Very fast prediction

Weaknesses

- Greedy - not optimal
- Prone to overfitting
- Orthogonal Boundaries -¿ sensitive to rotation (*Use PCA*)
- Unstable, very sensitive to small variations in training data (*high variance*)

Bagging (Bootstrap Aggregating)

Train B trees on bootstrap samples of size m drawn **with replacement** from the training set. Regression averages predictions; classification takes a majority vote:

$$\hat{f}_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}_b(x), \qquad \hat{y} = \arg\max_k \sum_{b=1}^{B} \mathbf{1}\{\hat{f}_b(x) = k\}.$$

Soft voting instead averages predicted class probabilities. Trees are high-variance; averaging reduces variance. If individual tree variance is σ^2 and pairwise correlation is ρ:

$$\text{Var}(\bar{f}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

Out-of-bag (OOB): A sample is omitted from one bootstrap sample with probability $(1-1/m)^m \to e^{-1} \approx 0.368$. Evaluate a training point using only trees that omitted it, providing an OOB performance estimate.

Random Forests

Random Forest = bagging + random feature selection at each split. Consider only a random subset of features ($max_features$), decorrelating trees so averaging reduces variance more effectively. Slide heuristics: $\sqrt{n}$ features per classification split; about $n/3$ or n per regression split (n total features). **Extra-Trees:** Also randomize split thresholds. Adding trees generally improves then stabilizes performance; it reduces the averaging-variance term rather than making a single tree more complex. OOB estimates can track test accuracy.

Mean Decrease in Impurity (MDI): Sum impurity reductions of splits using feature j, weighted by the fraction of samples reaching each split, then average over the trees and normalize.

$$\text{Imp}(j) \propto \frac{1}{B}\sum_{b=1}^{B} \sum_{\substack{t \in T_b \\ k(t)=j}} \frac{m_t}{m}\Delta G_t.$$

Used to assess relative feature importance.

Week 5/6 - Deeper into Bootstrapping

Estimating uncertainty by resampling the data that we already have

Statistics - Finding reliability of an estimate

Observe a sample x, drawn i.i.d from an unknown distribution F. We can compute a statistic $\hat{\theta} = s(x)$, estimating a parameter $\theta = t(F)$: which can be a mean, median, correlation, regression coefficient, test accuracy. $\hat{\theta}$ is a random variable, from the *sampling distribution*. We would like to know:

- Standard error(SE): $SE(\hat{\theta}) = \sqrt{VAR_f(\hat{\theta})}$
- Bias: $Bias(\hat{\theta}) = \mathbb{E}_F[\hat{\theta}] - \hat{\theta}$
- Confidence interval (CI): a data-based range [L,U] with $P(L \leq \theta \leq U) \approx 1-\alpha$

However, there is no simple formula for most statistics, cannot just draw new samples from F

Fix / Idea

Replace the unknown distribution F with the **empirical distribution** $\hat{F}_m$:

$$\hat{F}_m(t) = \frac{1}{m}\sum_{i=1}^{m} \mathbf{1}\{x_i \leq t\}$$

- $\hat{F}_m$ assigns probability $1/m$ to each observation x_i.
- **Bootstrap sampling**: Draw m observations from $\hat{F}_m$ **with replacement**.
- Bootstrap sample: $x^* = (x_1^*, \ldots, x_m^*)$.
- As $m \to \infty$, $\hat{F}_m \to F$ (Glivenko–Cantelli theorem).

Real World vs. Bootstrap World:

Real World	Bootstrap World
F (unknown)	$\hat{F}_m$ (known)
$x \sim F$	$x^* \sim \hat{F}_m$
$\theta = t(F)$	$\hat{\theta} = t(\hat{F}_m)$
$\hat{\theta} = s(x)$	$\hat{\theta}^* = s(x^*)$
$\hat{\theta} - \theta$	$\hat{\theta}^* - \hat{\theta}$

Key Idea: The variability of $\hat{\theta}^*$ around $\hat{\theta}$ approximates the variability of $\hat{\theta}$ around θ.

Procedure:

- Draw indices $i_1, \ldots, i_m$ uniformly from $\{1, \ldots, m\}$ with replacement. This becomes $x^{*b} = (x_{i1}, \ldots, x_{im})$
- Compute the statistic $\hat{\theta_b}* = s(x^{*b})$
- Use the empirical distribution in place of the sampling distribution.

Bootstrap Estimates of Standard Error and Bias

Given B bootstrap estimates $\hat{\theta}_1^*, \ldots, \hat{\theta}_B^*$, define their mean:

$$\bar{\theta}^* = \frac{1}{B}\sum_{b=1}^{B} \hat{\theta}_b^*$$

Bootstrap Standard Error and Bias:

$$\widehat{\text{SE}}_{\text{boot}} = \sqrt{\frac{1}{B-1}\sum_{b=1}^{B}(\hat{\theta}_b^* - \bar{\theta}^*)^2}$$

$$\widehat{\text{Bias}}_{\text{boot}} = \bar{\theta}^* - \hat{\theta}$$

Bias-Corrected Estimate:

$$\hat{\theta}_{bc} = \hat{\theta} - \widehat{\text{Bias}}_{\text{boot}} = 2\hat{\theta} - \bar{\theta}^*$$

May increase variance.

Two Sources of Error:

- **Statistical error:** $\hat{F}_m \neq F$; decreases as sample size m increases.
- **Monte Carlo error:** Finite B bootstrap samples; decreases as B increases.

Ideal Bootstrap ($B \to \infty$): For the sample mean, the bootstrap SE equals the plug-in estimate:

$$\text{SE}_{\hat{F}_m}(\bar{x}^*) = \frac{\hat{\sigma}}{\sqrt{m}}, \qquad \hat{\sigma}^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \bar{x})^2$$

Bootstrap Confidence Intervals

Let $\hat{\theta}^*_{(\alpha)}$ denote the α-quantile of the bootstrap estimates. Three common $(1-\alpha)$ confidence intervals for θ:

- **Normal:** Assumes $\hat{\theta}$ is approximately Gaussian.

$$\hat{\theta} \pm z_{1-\alpha/2}\widehat{\text{SE}}_{\text{boot}}$$

- **Percentile:** Uses bootstrap quantiles directly; respects skewness and parameter bounds.

$$\left[\hat{\theta}^*_{(\alpha/2)}, \hat{\theta}^*_{(1-\alpha/2)}\right]$$

- **Basic (Pivotal):** Assumes $\hat{\theta}^* - \hat{\theta} \approx \hat{\theta} - \theta$.

$$\left[2\hat{\theta} - \hat{\theta}^*_{(1-\alpha/2)}, 2\hat{\theta} - \hat{\theta}^*_{(\alpha/2)}\right]$$

Refinements: BCa (bias-corrected and accelerated) and studentized bootstrap intervals account for bias and/or skewness, often improving accuracy at greater computational cost.

How many distinct points does a bootstrap sample contain?

Never drawn:

$$(1 - \frac{1}{m})^m \to_{m\to\infty} e^{-1} \approx 0.368$$

So each instance appears with 63.2 percent probability, rest 36.8 left out are *out-of-bag*(OOB) instances.

Bootstrap Applications in Machine Learning

Bagging and Random Forests:

- Train one model per bootstrap sample and aggregate predictions.

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*b}(x)$$

- Averaging reduces variance in unstable learners (e.g., deep decision trees).

Out-of-Bag (OOB) Error:

- Each training observation is excluded from approximately 37% of bootstrap samples.
- Predict using only models that excluded the observation to estimate generalization error without a separate validation set.

Model Evaluation:

- **Confidence Intervals:** Bootstrap the test set and recompute metrics (accuracy, AUC).
- **Model Comparison:** Bootstrap the difference in metrics using the same resampled test sets.
- **.632 Estimator:** Combines training and OOB errors:

$$\widehat{\text{Err}}_{.632} = 0.368\,\widehat{\text{Err}}_{\text{train}} + 0.632\,\widehat{\text{Err}}_{\text{OOB}}$$

Where Bootstrapping fails:

- Dependent data - time series or structure of grouped data -¿ use a block or cluster bootstrap
- For very small m or number of samples, the empirical distribution is a bad representation of the true sampling distribution
- For extreme statistics, a resample can never exceed these values
- Heavy tails with infinite variance

Week 5/6 - More information theory

Entropy is a measure of uncertainty, or lack of predictability. Can also be interpreted as the information content of a data source. Think high entropy of p, observations $X_n \sim p$ are hard to predict, meaning the dataset carries a lot of information

Entropy of a Discrete Random Variable

Entropy: Measures the uncertainty of a discrete random variable X with K possible outcomes.

$$H(X) = -\sum_{k=1}^{K} p(X=k)\log_2 p(X=k) = -\mathbb{E}_X[\log_2 p(X)]$$

- **Units:** Base-2 logarithm gives bits; natural logarithm gives nats.
- **Example:** $p = [0.25, 0.25, 0.2, 0.15, 0.15]$ gives $H(X) \approx 2.29$ bits.
- **Maximum Entropy:** Uniform distribution, $p(X=k) = 1/K$, gives $H(X) = \log_2 K$.
- **Minimum Entropy:** $H(X) = 0$ when one outcome has probability 1 (no uncertainty).

Think of a coin toss, the maximum value 1 bit = 0.5 (fair coin - needs exactly one yes/no question. If it is 0 or 1, it's almost deterministic and carries almost no information.

Estimating entropy is hard for large alphabets

Cross Entropy

$$H_{ce}(p,q) = -\sum_{k=1}^{K} p_k \log q_k$$

Expected number of bits to compress data from p using a code built for q. Minimized by setting $q = p$, giving $H(p)$, Shannon's source coding theorem, cannot compress below this.

Joint Entropy

Joint Entropy: Measures the total uncertainty of two random variables X and Y.

$$H(X,Y) = -\sum_{x,y} p(x,y)\log_2 p(x,y)$$

- **Bounds:**

$$\max\{H(X), H(Y)\} \le H(X,Y) \le H(X) + H(Y)$$

- **Independence:** $X \perp Y$ implies $H(X,Y) = H(X) + H(Y)$.
- **Dependence:** Reduces joint entropy below the sum of individual entropies.
- **Example:** Even/prime indicators over $\{1,\dots,8\}$ have $H(X) = H(Y) = 1$ bit, but $H(X,Y) \approx 1.81$ bits.

Conditional Entropy and Chain Rule

Conditional Entropy: Measures remaining uncertainty in Y given X.

$$H(Y \mid X) = \mathbb{E}_{p(X)}[H(p(Y \mid X))] = H(X,Y) - H(X)$$

- **Deterministic:** If $Y = f(X)$, then $H(Y \mid X) = 0$.
- **Independent:** If $X \perp Y$, then $H(Y \mid X) = H(Y)$.
- **Conditioning reduces entropy (on average):**

$$H(Y \mid X) \le H(Y)$$

Chain Rule for Entropy:

$$H(X_1,\dots,X_n) = \sum_{i=1}^{n} H(X_i \mid X_1,\dots,X_{i-1})$$

For two variables:

$$H(X,Y) = H(X) + H(Y \mid X) = H(Y) + H(X \mid Y)$$

Worked Example

$$H(X_1,X_2,X_3) = H(X_1) + H(X_2 \mid X_1) + H(X_3 \mid X_1, X_2)$$

If X_i are drawn iid, this collapses to $3H(X_1)$

Perplexity

Perplexity: Measures uncertainty as the effective number of equally likely outcomes.

$$\text{Perplexity}(p) = 2^{H(p)}$$

- **Uniform distribution:** K equally likely states give perplexity K.
- **Minimum:** Perplexity = 1 (perfect predictability).
- **Language Models:** Evaluated using cross-entropy:

$$\text{Perplexity}(p_D, q) = 2^{H_{\text{CE}}(p_D,q)}$$

  where p_D is the empirical data distribution and q is the model.
- **Interpretation:** Weighted average branching factor (effective number of possible next tokens).
- **Lower Bound:** Since $H(p^*) \le H_{\text{CE}}(p^*, q)$, model perplexity cannot fall below $2^{H(p^*)}$.

Differential Entropy (Continuous)

Differential Entropy: Continuous analogue of discrete entropy.

$$h(X) = -\int_{\mathcal{X}} p(x)\log_2 p(x)\,dx$$

- **Uniform Distribution:** If $X \sim \text{Unif}(0,a)$, then $h(X) = \log_2 a$.
- **Can Be Negative:** Unlike discrete entropy, e.g., $X \sim \text{Unif}(0, 1/8)$ gives $h(X) = -3$ bits.
- **Quantization:** For bins of width 2^{-n}, discrete entropy is approximately $h(X) + n$ bits (under suitable regularity conditions).

Entropy of a Gaussian

Multivariate Gaussian Entropy: For $X \sim \mathcal{N}(\mu, \Sigma)$ with dimension d,

$$h(X) = \frac{1}{2}\ln|2\pi e\Sigma| = \frac{d}{2} + \frac{d}{2}\ln(2\pi) + \frac{1}{2}\ln|\Sigma|$$

1D Gaussian:

$$h(\mathcal{N}(\mu,\sigma^2)) = \frac{1}{2}\ln(2\pi e\sigma^2)$$

- **Entropy vs. Variance:** Entropy increases monotonically with variance for a single Gaussian, but not necessarily for arbitrary distributions.
- **Example:** A bimodal mixture can have increasing variance as its modes separate while entropy remains approximately constant.
- Gaussian entropy depends on covariance, not the mean.

Estimating Differential Entropy via Discretization

- Approximate differential entropy by binning continuous data and estimating probabilities.
- **Scott's Rule:** Approximate number of bins:

$$B = N^{1/3} \frac{\max(D) - \min(D)}{3.5\,\sigma(D)}$$

- **Limitation:** Binning scales poorly in high dimensions (curse of dimensionality).

KL Divergence (Relative Entropy)

KL Divergence: Measures how different two probability distributions p and q are.

$$D_{\mathrm{KL}}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = H_{\mathrm{CE}}(p,q) - H(p)$$

- $D_{\mathrm{KL}}(p\|q) \geq 0$, with equality iff $p = q$.
- Not symmetric: $D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$ in general.
- Measures extra coding cost from using q instead of p.
- **Information Inequality:** Non-negativity follows from Jensen's inequality.
- **Maximum Entropy:** Uniform distribution maximizes entropy, $H(X) \leq \log |\mathcal{X}|$.

KL Divergence and MLE: Minimizing KL divergence from the empirical distribution to a model is equivalent to maximizing likelihood.

Forward vs. Reverse KL:

- **Forward KL:** $D_{\mathrm{KL}}(p\|q)$ encourages mode-covering (zero-avoiding).
- **Reverse KL:** $D_{\mathrm{KL}}(q\|p)$ encourages mode-seeking (zero-forcing); used in variational inference.

Mutual Information (MI)

Mutual Information: Measures how much knowing one variable reduces uncertainty about another.

$$I(X;Y) = H(X) - H(X \mid Y) = H(Y) - H(Y \mid X)$$

$$I(X;Y) = H(X) + H(Y) - H(X,Y)$$

- $I(X;Y) \geq 0$; equals 0 iff $X \perp Y$.
- Symmetric: $I(X;Y) = I(Y;X)$.
- Equivalent to KL divergence between the joint distribution and the product of marginals.
- **Conditional MI:** Additional information shared by X and Y after conditioning on Z:

$$I(X;Y \mid Z) = H(X \mid Z) - H(X \mid Y,Z)$$

- **Chain Rule:** MI can be decomposed into successive conditional MI terms.

Applications and Properties of MI

- **Generalized Correlation:** MI detects dependencies beyond linear correlation.
- **Normalized MI:**

$$\mathrm{NMI}(X,Y) = \frac{I(X;Y)}{\min(H(X), H(Y))} \in [0,1]$$

- **MIC:** Maximal Information Coefficient; detects nonlinear relationships.
- **MICe / TICe:** Efficient estimators for measuring and detecting relationships.

Information-Theoretic Inequalities

Data Processing Inequality: For a Markov chain $X \to Y \to Z$,

$$I(X;Z) \leq I(X;Y)$$

Processing data cannot create additional information about the original variable.

- **Sufficient Statistic:** $s(D)$ preserves all information in dataset D relevant to parameter θ.
- **Minimal Sufficient Statistic:** Maximum compression without losing information about θ.
- **Fano's Inequality:** Relates conditional entropy to a lower bound on classification error.

Additional Information Theory Results

- **KL and χ^2:** For similar distributions, KL divergence is approximately half the χ^2 divergence (using natural logarithms).
- **Plug-in Entropy Estimation:** Estimate entropy using empirical probabilities $\hat{p}$ instead of the true p.
- **Finite-Sample Bias:** Plug-in entropy is biased downward, especially when the number of possible outcomes is large relative to sample size.
- **Improvement:** Smoothing or Bayesian estimators can reduce estimation bias.

Week 7 - Ensemble Learning

An ensemble is a group of predictors whose outputs are aggregated into a single prediction, usually beeter than the best individual predictor. Needs two ingredients:

- Enough predictors
- Diverse predictors (those that make different errors)

Voting Classifiers

Hard voting: each classifier casts one vote, the ensemble predicts the majority class. Even if the classifiers are weak, the ensemble can be strong if the two ingredients are met.

Works via *the law of large numbers.*

Law of Large Numbers As the observed number of samples increases, the series/probability will stabilize at the true probability. **As long as the they are independent, our classifiers are not**

Soft Voting: Each classifier estimates class probabilities, the ensemble averages them and predicts the class with the highest average probability.

Train predictors as independent/diverse as possible, on different training data

Bagging and Pasting

Use the same algorithm for every predictor, but train each on a different random subset of the training set.

Bagging: Stands for bootstrap aggregating, sampling with replacement.

Pasting: Sampling without replacement. Instance can be sampled several times across predictors but **not for the same predictor**.

Aggregation

Classification: the **statistical mode** - the most frequent Regression: the **average** of the predictions

Each individual predictor has higher bias, but aggregation reduces bias and variance, usually giving the ensemble a similar bias then a single predictor **but a lower variance**.

Bagging vs Pasting

Bagging is generally preferred since it introduces more diversity with each predictor, resulting in less variance. For both use **cross-validation** to evaluate and select the one that works best.

	Bagging	Pasting
Sampling	with replacement	without replacement
Diversity of subsets	higher	lower
Bias of ensemble	slightly higher	slightly lower
Correlation of predictors	lower	higher
Variance of ensemble	lower	higher

Random patches, Random subspaces, Extra-trees

Random Patches: random sampling of **both** training instances and features Random Subspaces: Random sampling of **features only** Extra Trees (Extremely Randomized Trees): Use *random thresholds* for each feature, rather than searching for the best thresholds

Random Forests

An ensemble of Decision Trees, trained via the bagging method (or sometimes pasting). Introduces more randomness:

- Search for best feature **among a random subset of features**

Results in more diversity(more bias less variance), and yields an overall better model

Deciding Random Forest vs Extra-Trees? - compare both using cross-validation, tune hyperparameters using grid search (evaluates all combinations of hyperparameters with cross-validation)

Feature Importance: tend to be closer to root in a decision tree, use average depth to estimate its importance.

Random Forests are a handy tool for feature selection this way.

Boosting

Any ensemble method that can combine several weak learners into a strong learner. Train them **sequentially** and have each trying to correct its predecessor.

13.7.1 AdaBoost

AdaBoost is short for adaptive boosting, it is called this because it automatically adjusts to the weak learners' actual error rates as it runs, so no such prior knowledge is needed. It does this in two ways:

- **It reweights the training examples each round**. Examples the previous classifiers got wrong are given more weight.
- **It weights each weak learner's vote by its accuracy** More accurate classifiers count for more in the final

combined prediction.

So it boosts weak learners into a strong one, and it adapts to how well each of them performs as it goes. **Similar to gradient descent, but it adds predictors instead of single parameters Acts like bagging/pasting except that the predictors have different weights depending on their overall accuracy Drawback - Cannot be fully parallelized, does not scale as well**

1. Initialize Weights: For m training instances,

$$w^{(i)} = \frac{1}{m}, \quad i = 1, \ldots, m$$

2. Weighted Error Rate: For predictor j,

$$r_j = \frac{\sum_{i:\hat{y}_j^{(i)} \neq y^{(i)}} w^{(i)}}{\sum_{i=1}^{m} w^{(i)}}$$

3. Predictor Weight:

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j}$$

where η is the learning rate (default 1).

- r_j small $\Rightarrow$ large α_j (accurate).
- $r_j \approx 0.5 \Rightarrow \alpha_j \approx 0$ (random).
- $r_j > 0.5 \Rightarrow \alpha_j < 0$ (worse than random).

4. Update Instance Weights:

$$w^{(i)} \leftarrow \begin{cases} w^{(i)}, & \hat{y}_j^{(i)} = y^{(i)} \\ w^{(i)} e^{\alpha_j}, & \hat{y}_j^{(i)} \neq y^{(i)} \end{cases}$$

5. Normalize Weights:

$$w^{(i)} \leftarrow \frac{w^{(i)}}{\sum_{k=1}^{m} w^{(k)}}$$

6. Repeat: Train the next predictor using updated weights until the desired number of predictors is reached or a perfect predictor is found.

Weighted Voting: For a new instance x, each of the N predictors votes for a class, weighted by its predictor weight α_j.

$$\hat{y}(x) = \arg\max_k \sum_{\substack{j=1 \\ \hat{y}_j(x)=k}}^{N} \alpha_j$$

- The predicted class receives the largest sum of weighted votes.
- More accurate predictors have greater influence due to larger α_j.

Example: Three predictors with weights $(1.2, 0.4, 0.7)$ vote (A, B, B).

$$A : 1.2, \qquad B : 0.4 + 0.7 = 1.1$$

Thus, $\hat{y} = A$, despite B receiving more unweighted votes.

13.7.2 Gradient Boosting

Also works sequentially, instead each new predictor is fit to the **residual errors** made by the previous predictor

1. Train first tree on the original targets y:
$$h_1(x) = \text{pred}^{(1)}(x)$$
2. Train second tree on residuals:
$$r_1 = y - h_1(x)$$
3. Train third tree on the remaining residuals:
$$r_2 = r_1 - h_2(x)$$
4. Combine predictions by summation:
$$\hat{y}(x) = \sum_{i=1}^{N} h_i(x)$$

Key Idea: Each new tree learns to correct the errors remaining from the previous ensemble.

Number of Trees:

- Too few trees $\Rightarrow$ **underfitting**.
- Too many trees $\Rightarrow$ **overfitting**.
- Select the number of trees using validation performance or **early stopping.**

13.7.3 Stacking

Instead of using trivial (ad-hoc) functions such as hard voting to aggregate the predictions of an ensemble, **train a model to perform the aggregation**. This final model is called a **blender**. Or meta-learner.

Training the blender

1. Split the training set into two subsets.
2. Use subset 1 to train the first-layer predictors
3. Use them to make predictions on subset 2 (the held-out) set. Predictions are "clean", instances not seen before, each one gets three predicted values
4. Build a new training set: the three predictions are the input features, original targets are kept
5. Train the blender on this new set: it learns to predict the target from the first layer's predictions

Multi-layer stacking

Several different blenders can be trained, called a layer of blenders. Split the training set into three subsets

1. Train the first layer
2. Builds the training set for the second layer, from first layer predictions
3. Build the training set for the third layer, from second-layer predictions

A prediction for a new instance goes *through each layer sequentially.*

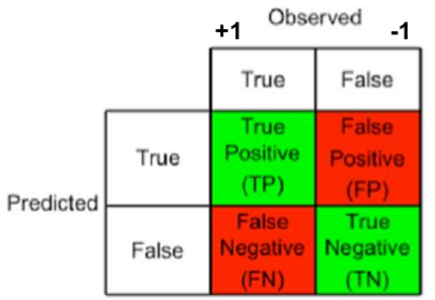
Week 7 - Machine Learning Metrics

Binary Classification Metrics

Four possible situations w.r.t observed known and predicted:

- True Positive **TP**
- True Negative **TN**
- False Positive **FP**
- False Negative **FN**

A confusion metric is a graphic that shows this:



Measures based on these:

Accuracy: Ratio of instances correctly classified

$$accuracy = \frac{correct\ classifications}{\#instances} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision: Accuracy with respect to instances predicted as positives

$$Precision = \frac{TP}{TP + FP}$$

Recall: Ratio of positive instances that are correctly predicted

$$recall = \frac{TP}{TP + FN}$$

Disadvantages: Accuracy-Precision-Recall all depend on choice of test set, sensitive to class skew, produce only a scalar as an assessment of the model

ROC Curves

These are independent of the test data, at least w.r.t to class skew. Require *instance prediction-confidence* for its construction. Each ROC point is given by false positive rate, true positive rate.

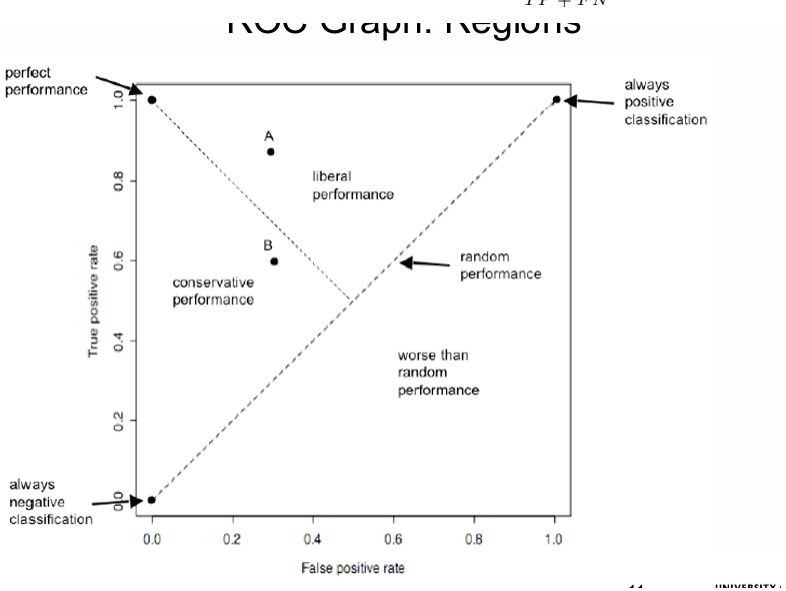
FPR/TPR

- False positive rate - How many negatives perceived positive - minimize

$$false_positive_rate = \frac{FP}{TN + FP}$$

- True positive rate - How many positives perceived positive - maximize

$$true_positive_rate = \frac{TP}{TP + FN}$$



Construction - Normalize all confidence values, between 0,1. Enumerate different decision thresholds and find ROC points in the process.

F-Beta Score (F_β)

The F_β score combines precision (P) and recall (R) using a weighted harmonic mean:

$$F_\beta(P, R) = \frac{(1 + \beta^2)PR}{\beta^2 P + R}$$

β controls the relative importance of precision and recall:

- $\beta < 1$: Prioritizes precision.
- $\beta = 1$: Equal importance (F1 score).
- $\beta > 1$: Prioritizes recall.

Special Cases:

- $\beta = 1$: $F_1 = \frac{2PR}{P + R}$ (harmonic mean).
- $\beta^2 = 0.5$: $F_\beta = \frac{3PR}{P + 2R}$.
- $\beta = 0.5$: $F_{0.5} = \frac{5PR}{P + 4R}$.
- $\beta = 0$: $F_0 = P$.
- $\beta \to \infty$: $F_\beta \to R$.

Week 8/9 - Bayesian Learning

Two Roles for Bayesian Methods

- Practical Learning Algorithms (Naive Bayes, Bayesian Belief, Combine prior knowledge with observed data)
- Useful Conceptual Framework (gold standard, occams razor, characterize probabilistic algorithms)

Bayes Theorem

Idea: Infer something about X given knowledge of Y.

$$\Pr(h \mid D) = \frac{\Pr(D \mid h)\Pr(h)}{\Pr(D)}$$

where D is the set of training examples for a concept and h is a candidate hypothesis. **Terms**

- $\Pr(h)$ - prior probability of hypothesis h
- $\Pr(D)$ - prior probability of training data D
- $\Pr(h \mid D)$ - posterior probability of h given D
- $\Pr(D \mid h)$ - probability of D given h (the likelihood)

$$\underbrace{P(h \mid D)}_{\text{posterior}} = \frac{\overbrace{P(D \mid h)}^{\text{likelihood}} \cdot \overbrace{P(h)}^{\text{prior}}}{\underbrace{P(D)}_{\text{evidence}}}$$

- $P(h \mid D)$ increases with $P(h)$: a priori plausible hypotheses are favored.
- $P(h \mid D)$ increases with $P(D \mid h)$: hypotheses that explain the data well are favored.
- $P(h \mid D)$ decreases with $P(D)$: data that are likely regardless of h provide little evidence for h.
- $P(D)$ does not depend on h: it is the same normalizing constant for every hypothesis.

MAP Hypothesis

Most probable hypothesis given the training data: **Maximum a Posteriori**

$$\begin{aligned} h_{map} &= \arg\max_{h \in H} \Pr(h \mid D) \\ &= \arg\max_{h \in H} \frac{\Pr(D \mid h)\Pr(h)}{P(D)} \\ &= \arg\max_{h \in H} \Pr(D \mid h)\Pr(h) \end{aligned}$$

We drop $P(D)$ is dropped because it is a positive constant independent of h.

Maximum Likelihood (ML) Hypothesis

Assume a uniform prior on the hypothesis space:

$$\forall h \in H: \qquad \Pr(h) = \frac{1}{|H|}$$

then $P(H)$ is also a constant and (1) simplifies to,

$$h_{MAP/ML} = \arg\max_{h \in H} \Pr(D \mid h)$$

where $P(D \mid h)$ is called the likelihood

Basic Formulas for Probabilities

Product Rule: Probability of a conjunction of two events A and B.

$$P(A \wedge B) = P(A \mid B)P(B) = P(B \mid A)P(A)$$

Sum Rule: Probability of a disjunction of two events A and B.

$$P(A \vee B) = P(A) + P(B) - P(A \wedge B)$$

Theorem of Total Probability: If $A_1, \ldots, A_n$ are mutually exclusive and exhaustive, with $\sum_{i=1}^{n} P(A_i) = 1$, then

$$P(B) = \sum_{i=1}^{n} P(B \mid A_i)P(A_i)$$

Note: Bayes' theorem follows directly from the product rule. The theorem of total probability provides $P(D)$, the evidence (normalizing constant) in Bayes' theorem.

$$f_m(h)^2 = r\left(\ln\frac{2}{r} + 1\right) - \frac{\ln(\frac{\eta}{4})}{m}$$